

Gym Membership Management System

Tables

Members

```
CREATE TABLE Members (  
  member_id INT PRIMARY KEY,  
  member_name VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE,  
  phone VARCHAR(15),  
  join_date DATE  
);
```

```
MySQL localhost:3306 ssl gym_membership_management_system SQL > CREATE TABLE Members (  
-> member_id INT PRIMARY KEY,  
-> member_name VARCHAR(100) NOT NULL,  
-> email VARCHAR(100) UNIQUE,  
-> phone VARCHAR(15),  
-> join_date DATE  
-> );  
  
Query OK, 0 rows affected (0.0192 sec)  
MySQL localhost:3306 ssl gym_membership_management_system SQL > desc members;  
  
+-----+-----+-----+-----+-----+-----+  
| Field | Type | Null | Key | Default | Extra |  
+-----+-----+-----+-----+-----+-----+  
| member_id | int | NO | PRI | NULL | |  
| member_name | varchar(100) | NO | | NULL | |  
| email | varchar(100) | YES | UNI | NULL | |  
| phone | varchar(15) | YES | | NULL | |  
| join_date | date | YES | | NULL | |  
+-----+-----+-----+-----+-----+-----+  
5 rows in set (0.0059 sec)
```

Trainers

```
CREATE TABLE Trainers (  
  trainer_id INT PRIMARY KEY,  
  trainer_name VARCHAR(100),  
  specialization VARCHAR(100),  
  phone VARCHAR(15)  
);
```

```
MySQL localhost:3306 ssl gym_membership_management_system SQL > CREATE TABLE Trainers (
    -> trainer_id INT PRIMARY KEY,
    -> trainer_name VARCHAR(100),
    -> specialization VARCHAR(100),
    -> phone VARCHAR(15)
    -> );

Query OK, 0 rows affected (0.0153 sec)
MySQL localhost:3306 ssl gym_membership_management_system SQL > desc Trainers;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| trainer_id | int           | NO   | PRI | NULL    |       |
| trainer_name | varchar(100) | YES  |     | NULL    |       |
| specialization | varchar(100) | YES  |     | NULL    |       |
| phone      | varchar(15)  | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.0034 sec)
```

Plans

```
CREATE TABLE Plans (
    plan_id INT PRIMARY KEY,
    plan_name VARCHAR(100),
    duration INT,
    price DECIMAL(10,2)
);
```

```
MySQL localhost:3306 ssl gym_membership_management_system SQL > CREATE TABLE Plans (
    -> plan_id INT PRIMARY KEY,
    -> plan_name VARCHAR(100),
    -> duration INT,
    -> price DECIMAL(10,2)
    -> );

Query OK, 0 rows affected (0.0142 sec)
MySQL localhost:3306 ssl gym_membership_management_system SQL > desc Plans;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| plan_id    | int           | NO   | PRI | NULL    |       |
| plan_name  | varchar(100) | YES  |     | NULL    |       |
| duration   | int           | YES  |     | NULL    |       |
| price      | decimal(10,2) | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.0024 sec)
```

Memberships

```
CREATE TABLE Memberships (
    membership_id INT PRIMARY KEY,
    member_id INT,
    plan_id INT,
    start_date DATE,
    end_date DATE,
    FOREIGN KEY (member_id) REFERENCES Members(member_id),
    FOREIGN KEY (plan_id) REFERENCES Plans(plan_id)
);
```

```
MySQL localhost:3306 ssl gym_membership_management_system SQL > CREATE TABLE Memberships (
-> membership_id INT PRIMARY KEY,
-> member_id INT,
-> plan_id INT,
-> start_date DATE,
-> end_date DATE,
-> FOREIGN KEY (member_id) REFERENCES Members(member_id),
-> FOREIGN KEY (plan_id) REFERENCES Plans(plan_id)
-> );

Query OK, 0 rows affected (0.0253 sec)
MySQL localhost:3306 ssl gym_membership_management_system SQL > desc Memberships;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| membership_id | int | NO | PRI | NULL | |
| member_id | int | YES | MUL | NULL | |
| plan_id | int | YES | MUL | NULL | |
| start_date | date | YES | | NULL | |
| end_date | date | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.0028 sec)
```

Payments

```
CREATE TABLE Payments (
payment_id INT PRIMARY KEY,
member_id INT,
amount DECIMAL(10,2),
payment_date DATE,
status VARCHAR(20),
FOREIGN KEY (member_id) REFERENCES Members(member_id)
);
```

```
MySQL localhost:3306 ssl gym_membership_management_system SQL > CREATE TABLE Payments (
-> payment_id INT PRIMARY KEY,
-> member_id INT,
-> amount DECIMAL(10,2),
-> payment_date DATE,
-> status VARCHAR(20),
-> FOREIGN KEY (member_id) REFERENCES Members(member_id)
-> );

Query OK, 0 rows affected (0.0192 sec)
MySQL localhost:3306 ssl gym_membership_management_system SQL > desc Payments;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| payment_id | int | NO | PRI | NULL | |
| member_id | int | YES | MUL | NULL | |
| amount | decimal(10,2) | YES | | NULL | |
| payment_date | date | YES | | NULL | |
| status | varchar(20) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.0030 sec)
```

- Insert sample data into all tables.

TABLE: *Members*

```
INSERT INTO Members (member_id, member_name, email, phone,
join_date) VALUES
```

(1, 'Aarav Sharma', 'aarav@gmail.com', '9876543210', '2024-01-10'),
(2, 'Diya Reddy', 'diya@gmail.com', '9123456780', '2024-01-15'),
(3, 'Rohan Kumar', 'rohan@gmail.com', '9012345678', '2024-02-01'),
(4, 'Sneha Patel', 'sneha@gmail.com', '9988776655', '2024-02-10'),
(5, 'Arjun Mehta', 'arjun@gmail.com', '9871234567', '2024-02-20'),
(6, 'Priya Singh', 'priya@gmail.com', '9765432109', '2024-03-01'),
(7, 'Karan Verma', 'karan@gmail.com', '9654321098', '2024-03-05'),
(8, 'Ananya Das', 'ananya@gmail.com', '9543210987', '2024-03-15'),
(9, 'Vikram Rao', 'vikram@gmail.com', '9432109876', '2024-03-25'),
(10, 'Neha Gupta', 'neha@gmail.com', '9321098765', '2024-04-01'),
(11, 'Rahul Nair', 'rahul@gmail.com', '9210987654', '2024-04-10'),
(12, 'Pooja Iyer', 'pooja@gmail.com', '9109876543', '2024-04-18'),
(13, 'Aditya Joshi', 'aditya@gmail.com', '9098765432', '2024-04-25'),
(14, 'Meera Kapoor', 'meera@gmail.com', '8987654321', '2024-05-01'),
(15, 'Siddharth', 'siddharth@gmail.com', '8876543210', '2024-05-10'),
(16, 'Kavya Nair', 'kavya@gmail.com', '8765432109', '2024-05-18'),
(17, 'Harsha Reddy', 'harsha@gmail.com', '8654321098', '2024-05-25'),
(18, 'Tanvi Shah', 'tanvi@gmail.com', '8543210987', '2024-06-01'),
(19, 'Manish Yadav', 'manish@gmail.com', '8432109876', '2024-06-10'),
(20, 'Ishita Jain', 'ishita@gmail.com', '8321098765', '2024-06-20');

Query:

```
MySQL localhost:3306 ssl gym_membership_management_system SQL > select * from Members;
```

member_id	member_name	email	phone	join_date
1	Aarav Sharma	aarav@gmail.com	9876543210	2024-01-10
2	Diya Reddy	diya@gmail.com	9123456780	2024-01-15
3	Rohan Kumar	rohan@gmail.com	9012345678	2024-02-01
4	Sneha Patel	sneha@gmail.com	9988776655	2024-02-10
5	Arjun Mehta	arjun@gmail.com	9871234567	2024-02-20
6	Priya Singh	priya@gmail.com	9765432109	2024-03-01
7	Karan Verma	karan@gmail.com	9654321098	2024-03-05
8	Ananya Das	ananya@gmail.com	9543210987	2024-03-15
9	Vikram Rao	vikram@gmail.com	9432109876	2024-03-25
10	Neha Gupta	neha@gmail.com	9321098765	2024-04-01
11	Rahul Nair	rahul@gmail.com	9210987654	2024-04-10
12	Pooja Iyer	pooja@gmail.com	9109876543	2024-04-18
13	Aditya Joshi	aditya@gmail.com	9098765432	2024-04-25
14	Meera Kapoor	meera@gmail.com	8987654321	2024-05-01
15	Siddharth Roy	siddharth@gmail.com	8876543210	2024-05-10
16	Kavya Nair	kavya@gmail.com	8765432109	2024-05-18
17	Harsha Reddy	harsha@gmail.com	8654321098	2024-05-25
18	Tanvi Shah	tanvi@gmail.com	8543210987	2024-06-01
19	Manish Yadav	manish@gmail.com	8432109876	2024-06-10
20	Ishita Jain	ishita@gmail.com	8321098765	2024-06-20

20 rows in set (0.0012 sec)

TABLE: Trainers

INSERT INTO Trainers (trainer_id, trainer_name, specialization, phone)
VALUES

(1, 'Rajesh Kumar', 'Weight Training', '9876501234'),

(2, 'Anita Sharma', 'Yoga', '9876502345'),

(3, 'Vikram Singh', 'Cardio Training', '9876503456'),

(4, 'Sneha Reddy', 'Zumba', '9876504567'),

(5, 'Arjun Patel', 'Bodybuilding', '9876505678'),

(6, 'Priya Nair', 'Aerobics', '9876506789'),

(7, 'Karan Mehta', 'CrossFit', '9876507890'),

(8, 'Neha Gupta', 'Pilates', '9876508901'),

(9, 'Rohit Verma', 'Strength Training', '9876509012'),
 (10, 'Pooja Das', 'Yoga Therapy', '9876510123'),
 (11, 'Siddharth Roy', 'HIIT', '9876511234'),
 (12, 'Meera Kapoor', 'Dance Fitness', '9876512345'),
 (13, 'Aditya Joshi', 'Functional Training', '9876513456'),
 (14, 'Kavya Iyer', 'Meditation', '9876514567'),
 (15, 'Harsha Rao', 'Endurance Training', '9876515678'),
 (16, 'Tanvi Shah', 'Kickboxing', '9876516789'),
 (17, 'Manish Yadav', 'Powerlifting', '9876517890'),
 (18, 'Ishita Jain', 'Flexibility Training', '9876518901'),
 (19, 'Rahul Khanna', 'Sports Training', '9876519012'),
 (20, 'Divya Agarwal', 'Personal Training', '9876520123');

Query:

```
MySQL localhost:3306 ssl gym_membership_management_system SQL > select * from Trainers;
```

trainer_id	trainer_name	specialization	phone
1	Rajesh Kumar	Weight Training	9876501234
2	Anita Sharma	Yoga	9876502345
3	Vikram Singh	Cardio Training	9876503456
4	Sneha Reddy	Zumba	9876504567
5	Arjun Patel	Bodybuilding	9876505678
6	Priya Nair	Aerobics	9876506789
7	Karan Mehta	CrossFit	9876507890
8	Neha Gupta	Pilates	9876508901
9	Rohit Verma	Strength Training	9876509012
10	Pooja Das	Yoga Therapy	9876510123
11	Siddharth Roy	HIIT	9876511234
12	Meera Kapoor	Dance Fitness	9876512345
13	Aditya Joshi	Functional Training	9876513456
14	Kavya Iyer	Meditation	9876514567
15	Harsha Rao	Endurance Training	9876515678
16	Tanvi Shah	Kickboxing	9876516789
17	Manish Yadav	Powerlifting	9876517890
18	Ishita Jain	Flexibility Training	9876518901
19	Rahul Khanna	Sports Training	9876519012
20	Divya Agarwal	Personal Training	9876520123

20 rows in set (0.0011 sec)

TABLE: Plans

INSERT INTO Plans (plan_id, plan_name, duration, price) VALUES

(1, 'Basic Monthly', 1, 1000.00),
(2, 'Standard Monthly', 1, 1500.00),
(3, 'Premium Monthly', 1, 2000.00),
(4, 'Basic Quarterly', 3, 2800.00),
(5, 'Standard Quarterly', 3, 4000.00),
(6, 'Premium Quarterly', 3, 5500.00),
(7, 'Basic Half-Yearly', 6, 5000.00),
(8, 'Standard Half-Yearly', 6, 7000.00),
(9, 'Premium Half-Yearly', 6, 9000.00),
(10, 'Basic Yearly', 12, 9000.00),
(11, 'Standard Yearly', 12, 12000.00),
(12, 'Premium Yearly', 12, 15000.00),
(13, 'Student Monthly', 1, 800.00),
(14, 'Student Quarterly', 3, 2200.00),
(15, 'Student Yearly', 12, 7000.00),
(16, 'Couple Monthly', 1, 2500.00),
(17, 'Couple Quarterly', 3, 7000.00),
(18, 'Couple Yearly', 12, 20000.00),
(19, 'Weekend Plan', 1, 1200.00),
(20, 'Personal Training Plan', 1, 3000.00);

Query:

```
MySQL localhost:3306 ssl gym_membership_management_system SQL > select * from Plans;
```

plan_id	plan_name	duration	price
1	Basic Monthly	1	1000.00
2	Standard Monthly	1	1500.00
3	Premium Monthly	1	2000.00
4	Basic Quarterly	3	2800.00
5	Standard Quarterly	3	4000.00
6	Premium Quarterly	3	5500.00
7	Basic Half-Yearly	6	5000.00
8	Standard Half-Yearly	6	7000.00
9	Premium Half-Yearly	6	9000.00
10	Basic Yearly	12	9000.00
11	Standard Yearly	12	12000.00
12	Premium Yearly	12	15000.00
13	Student Monthly	1	800.00
14	Student Quarterly	3	2200.00
15	Student Yearly	12	7000.00
16	Couple Monthly	1	2500.00
17	Couple Quarterly	3	7000.00
18	Couple Yearly	12	20000.00
19	Weekend Plan	1	1200.00
20	Personal Training Plan	1	3000.00

20 rows in set (0.0008 sec)

TABLE: Memberships

INSERT INTO Memberships (membership_id, member_id, plan_id, start_date, end_date) VALUES

(1, 1, 1, '2024-01-10', '2024-02-10'),

(2, 2, 2, '2024-01-15', '2024-02-15'),

(3, 3, 3, '2024-02-01', '2024-03-01'),

(4, 4, 4, '2024-02-10', '2024-05-10'),

(5, 5, 5, '2024-02-20', '2024-05-20'),

(6, 6, 6, '2024-03-01', '2024-06-01'),

(7, 7, 7, '2024-03-05', '2024-09-05'),

(8, 8, 8, '2024-03-15', '2024-09-15'),

(9, 9, 9, '2024-03-25', '2024-09-25'),

(10, 10, 10, '2024-04-01', '2025-04-01'),

(11, 11, 11, '2024-04-10', '2025-04-10'),

(12, 12, 12, '2024-04-18', '2025-04-18'),
 (13, 13, 13, '2024-04-25', '2024-05-25'),
 (14, 14, 14, '2024-05-01', '2024-08-01'),
 (15, 15, 15, '2024-05-10', '2025-05-10'),
 (16, 16, 16, '2024-05-18', '2024-06-18'),
 (17, 17, 17, '2024-05-25', '2024-08-25'),
 (18, 18, 18, '2024-06-01', '2025-06-01'),
 (19, 19, 19, '2024-06-10', '2024-07-10'),
 (20, 20, 20, '2024-06-20', '2024-07-20');

Query:

```

MySQL localhost:3306 ssl gym_membership_management_system SQL > select * from Memberships;
+-----+-----+-----+-----+-----+
| membership_id | member_id | plan_id | start_date | end_date |
+-----+-----+-----+-----+-----+
| 1 | 1 | 1 | 2024-01-10 | 2024-02-10 |
| 2 | 2 | 2 | 2024-01-15 | 2024-02-15 |
| 3 | 3 | 3 | 2024-02-01 | 2024-03-01 |
| 4 | 4 | 4 | 2024-02-10 | 2024-05-10 |
| 5 | 5 | 5 | 2024-02-20 | 2024-05-20 |
| 6 | 6 | 6 | 2024-03-01 | 2024-06-01 |
| 7 | 7 | 7 | 2024-03-05 | 2024-09-05 |
| 8 | 8 | 8 | 2024-03-15 | 2024-09-15 |
| 9 | 9 | 9 | 2024-03-25 | 2024-09-25 |
| 10 | 10 | 10 | 2024-04-01 | 2025-04-01 |
| 11 | 11 | 11 | 2024-04-10 | 2025-04-10 |
| 12 | 12 | 12 | 2024-04-18 | 2025-04-18 |
| 13 | 13 | 13 | 2024-04-25 | 2024-05-25 |
| 14 | 14 | 14 | 2024-05-01 | 2024-08-01 |
| 15 | 15 | 15 | 2024-05-10 | 2025-05-10 |
| 16 | 16 | 16 | 2024-05-18 | 2024-06-18 |
| 17 | 17 | 17 | 2024-05-25 | 2024-08-25 |
| 18 | 18 | 18 | 2024-06-01 | 2025-06-01 |
| 19 | 19 | 19 | 2024-06-10 | 2024-07-10 |
| 20 | 20 | 20 | 2024-06-20 | 2024-07-20 |
+-----+-----+-----+-----+-----+
20 rows in set (0.0012 sec)
  
```

TABLE: *Payments*

INSERT INTO Payments (payment_id, member_id, amount,
 payment_date, status) VALUES

(1, 1, 1000.00, '2024-01-10', 'Paid'),
(2, 2, 1500.00, '2024-01-15', 'Paid'),
(3, 3, 2000.00, '2024-02-01', 'Paid'),
(4, 4, 2800.00, '2024-02-10', 'Paid'),
(5, 5, 4000.00, '2024-02-20', 'Paid'),
(6, 6, 5500.00, '2024-03-01', 'Paid'),
(7, 7, 5000.00, '2024-03-05', 'Paid'),
(8, 8, 7000.00, '2024-03-15', 'Paid'),
(9, 9, 9000.00, '2024-03-25', 'Paid'),
(10, 10, 9000.00, '2024-04-01', 'Paid'),
(11, 11, 12000.00, '2024-04-10', 'Paid'),
(12, 12, 15000.00, '2024-04-18', 'Paid'),
(13, 13, 800.00, '2024-04-25', 'Pending'),
(14, 14, 2200.00, '2024-05-01', 'Paid'),
(15, 15, 7000.00, '2024-05-10', 'Paid'),
(16, 16, 2500.00, '2024-05-18', 'Pending'),
(17, 17, 7000.00, '2024-05-25', 'Paid'),
(18, 18, 20000.00, '2024-06-01', 'Paid'),
(19, 19, 1200.00, '2024-06-10', 'Pending'),
(20, 20, 3000.00, '2024-06-20', 'Paid');

Query:

```
MySQL localhost:3306 ssl gym_membership_management_system SQL > select * from payments;
```

payment_id	member_id	amount	payment_date	status
1	1	1000.00	2024-01-10	Paid
2	2	1500.00	2024-01-15	Paid
3	3	2000.00	2024-02-01	Paid
4	4	2800.00	2024-02-10	Paid
5	5	4000.00	2024-02-20	Paid
6	6	5500.00	2024-03-01	Paid
7	7	5000.00	2024-03-05	Paid
8	8	7000.00	2024-03-15	Paid
9	9	9000.00	2024-03-25	Paid
10	10	9000.00	2024-04-01	Paid
11	11	12000.00	2024-04-10	Paid
12	12	15000.00	2024-04-18	Paid
13	13	800.00	2024-04-25	Pending
14	14	2200.00	2024-05-01	Paid
15	15	7000.00	2024-05-10	Paid
16	16	2500.00	2024-05-18	Pending
17	17	7000.00	2024-05-25	Paid
18	18	20000.00	2024-06-01	Paid
19	19	1200.00	2024-06-10	Pending
20	20	3000.00	2024-06-20	Paid

20 rows in set (0.0017 sec)

BASIC QUERIES

1. Display all member

```
MySQL localhost:3306 ssl gym_membership_management_system SQL > select * from Members;
```

member_id	member_name	email	phone	join_date
1	Aarav Sharma	aarav@gmail.com	9876543210	2024-01-10
2	Diya Reddy	diya@gmail.com	9123456780	2024-01-15
3	Rohan Kumar	rohan@gmail.com	9012345678	2024-02-01
4	Sneha Patel	sneha@gmail.com	9988776655	2024-02-10
5	Arjun Mehta	arjun@gmail.com	9871234567	2024-02-20
6	Priya Singh	priya@gmail.com	9765432109	2024-03-01
7	Karan Verma	karan@gmail.com	9654321098	2024-03-05
8	Ananya Das	ananya@gmail.com	9543210987	2024-03-15
9	Vikram Rao	vikram@gmail.com	9432109876	2024-03-25
10	Neha Gupta	neha@gmail.com	9321098765	2024-04-01
11	Rahul Nair	rahul@gmail.com	9210987654	2024-04-10
12	Pooja Iyer	pooja@gmail.com	9109876543	2024-04-18
13	Aditya Joshi	aditya@gmail.com	9098765432	2024-04-25
14	Meera Kapoor	meera@gmail.com	8987654321	2024-05-01
15	Siddharth Roy	siddharth@gmail.com	8876543210	2024-05-10
16	Kavya Nair	kavya@gmail.com	8765432109	2024-05-18
17	Harsha Reddy	harsha@gmail.com	8654321098	2024-05-25
18	Tanvi Shah	tanvi@gmail.com	8543210987	2024-06-01
19	Manish Yadav	manish@gmail.com	8432109876	2024-06-10
20	Ishita Jain	ishita@gmail.com	8321098765	2024-06-20

20 rows in set (0.0012 sec)

2. Show plans with price greater than 5000

```
MySQL localhost:3306 ssl gym_membership_management_system SQL > select * from plans where price > 5000;
```

plan_id	plan_name	duration	price
6	Premium Quarterly	3	5500.00
8	Standard Half-Yearly	6	7000.00
9	Premium Half-Yearly	6	9000.00
10	Basic Yearly	12	9000.00
11	Standard Yearly	12	12000.00
12	Premium Yearly	12	15000.00
15	Student Yearly	12	7000.00
17	Couple Quarterly	3	7000.00
18	Couple Yearly	12	20000.00

9 rows in set (0.0039 sec)

3. List members who joined after a specific date

```
MySQL localhost:3306 ssl gym_membership_management_system SQL > select * from members where join_date > '2024-03-01';
```

member_id	member_name	email	phone	join_date
7	Karan Verma	karan@gmail.com	9654321098	2024-03-05
8	Ananya Das	ananya@gmail.com	9543210987	2024-03-15
9	Vikram Rao	vikram@gmail.com	9432109876	2024-03-25
10	Neha Gupta	neha@gmail.com	9321098765	2024-04-01
11	Rahul Nair	rahul@gmail.com	9210987654	2024-04-10
12	Pooja Iyer	pooja@gmail.com	9109876543	2024-04-18
13	Aditya Joshi	aditya@gmail.com	9098765432	2024-04-25
14	Meera Kapoor	meera@gmail.com	8987654321	2024-05-01
15	Siddharth Roy	siddharth@gmail.com	8876543210	2024-05-10
16	Kavya Nair	kavya@gmail.com	8765432109	2024-05-18
17	Harsha Reddy	harsha@gmail.com	8654321098	2024-05-25
18	Tanvi Shah	tanvi@gmail.com	8543210987	2024-06-01
19	Manish Yadav	manish@gmail.com	8432109876	2024-06-10
20	Ishita Jain	ishita@gmail.com	8321098765	2024-06-20

14 rows in set (0.0007 sec)

4. Display active memberships

```
MySQL localhost:3306 ssl gym_membership_management_system SQL > select * from memberships where curdate() between start_date and end_date;
```

Empty set (0.0047 sec)

5. Show trainers with a specific specialization

```
MySQL localhost:3306 ssl gym_membership_management_system SQL > select * from trainers where specialization = 'yoga';
```

trainer_id	trainer_name	specialization	phone
2	Anita Sharma	Yoga	9876502345

1 row in set (0.0018 sec)

Joins

1. Show member names with their plans

```

MySQL localhost:3306 ssl gym_membership_management_system SQL> select m.member_name,p.plan_name from members m inner join member
ships ms on m.member_id = ms.member_id inner join plans p on ms.plan_id = p.plan_id;
+-----+-----+
| member_name | plan_name |
+-----+-----+
| Aarav Sharma | Basic Monthly |
| Diya Reddy | Standard Monthly |
| Rohan Kumar | Premium Monthly |
| Sneha Patel | Basic Quarterly |
| Arjun Mehta | Standard Quarterly |
| Priya Singh | Premium Quarterly |
| Karan Verma | Basic Half-Yearly |
| Ananya Das | Standard Half-Yearly |
| Vikram Rao | Premium Half-Yearly |
| Neha Gupta | Basic Yearly |
| Rahul Nair | Standard Yearly |
| Pooja Iyer | Premium Yearly |
| Aditya Joshi | Student Monthly |
| Meera Kapoor | Student Quarterly |
| Siddharth Roy | Student Yearly |
| Kavya Nair | Couple Monthly |
| Harsha Reddy | Couple Quarterly |
| Tanvi Shah | Couple Yearly |
| Manish Yadav | Weekend Plan |
| Ishita Jain | Personal Training Plan |
+-----+-----+
20 rows in set (0.0020 sec)

```

2. Display trainer with assigned members

Query:

```
select t.trainer_name, m.member_name from trainers t cross join members m
on t.specialization is not null;
```

Output:

This query displays all possible combinations of trainers and members. Since there is no direct relationship defined between trainers and members, a CROSS JOIN is used, resulting in a Cartesian product.

For example, each trainer is paired with every member in the system.

If there are 20 trainers and 20 members, the query generates 400 rows.

To maintain readability, only a sample of the output is shown.

```
MySQL localhost:3306 ssl gym_membership_management_system SQL> select t.trainer_name, m.member_name from trainers t cross join m
members m on t.specialization is not null;
```

trainer_name	member_name
Divya Agarwal	Aarav Sharma
Rahul Khanna	Aarav Sharma
Ishita Jain	Aarav Sharma
Manish Yadav	Aarav Sharma
Tanvi Shah	Aarav Sharma
Harsha Rao	Aarav Sharma
Kavya Iyer	Aarav Sharma
Aditya Joshi	Aarav Sharma
Meera Kapoor	Aarav Sharma
Siddharth Roy	Aarav Sharma
Pooja Das	Aarav Sharma
Rohit Verma	Aarav Sharma
Neha Gupta	Aarav Sharma
Karan Mehta	Aarav Sharma
Priya Nair	Aarav Sharma
Arjun Patel	Aarav Sharma
Sneha Reddy	Aarav Sharma
Vikram Singh	Aarav Sharma
Anita Sharma	Aarav Sharma
Rajesh Kumar	Aarav Sharma
Divya Agarwal	Diya Reddy
Rahul Khanna	Diya Reddy
Ishita Jain	Diya Reddy
Manish Yadav	Diya Reddy
Tanvi Shah	Diya Reddy
Harsha Rao	Diya Reddy
Kavya Iyer	Diya Reddy
Aditya Joshi	Diya Reddy

3. Show membership details with member and plan

```
MySQL localhost:3306 ssl gym_membership_management_system SQL> select m.member_name,p.plan_name,ms.start_date,ms.end_date
-> from memberships ms inner join members m on ms.member_id =
-> m.member_id
-> inner join plans p on ms.plan_id=p.plan_id;
```

member_name	plan_name	start_date	end_date
Aarav Sharma	Basic Monthly	2024-01-10	2024-02-10
Diya Reddy	Standard Monthly	2024-01-15	2024-02-15
Rohan Kumar	Premium Monthly	2024-02-01	2024-03-01
Sneha Patel	Basic Quarterly	2024-02-10	2024-05-10
Arjun Mehta	Standard Quarterly	2024-02-20	2024-05-20
Priya Singh	Premium Quarterly	2024-03-01	2024-06-01
Karan Verma	Basic Half-Yearly	2024-03-05	2024-09-05
Ananya Das	Standard Half-Yearly	2024-03-15	2024-09-15
Vikram Rao	Premium Half-Yearly	2024-03-25	2024-09-25
Neha Gupta	Basic Yearly	2024-04-01	2025-04-01
Rahul Nair	Standard Yearly	2024-04-10	2025-04-10
Pooja Iyer	Premium Yearly	2024-04-18	2025-04-18
Aditya Joshi	Student Monthly	2024-04-25	2024-05-25
Meera Kapoor	Student Quarterly	2024-05-01	2024-08-01
Siddharth Roy	Student Yearly	2024-05-10	2025-05-10
Kavya Nair	Couple Monthly	2024-05-18	2024-06-18
Harsha Reddy	Couple Quarterly	2024-05-25	2024-08-25
Tanvi Shah	Couple Yearly	2024-06-01	2025-06-01
Manish Yadav	Weekend Plan	2024-06-10	2024-07-10
Ishita Jain	Personal Training Plan	2024-06-20	2024-07-20

20 rows in set (0.0035 sec)

4. List members with payment details

```
MySQL localhost:3306 ssl gym_membership_management_system SQL -> SELECT m.member_name, p.amount, p.payment_date, p.status
-> FROM Members m
-> INNER JOIN Payments p ON m.member_id = p.member_id;
```

member_name	amount	payment_date	status
Aarav Sharma	1000.00	2024-01-10	Paid
Diya Reddy	1500.00	2024-01-15	Paid
Rohan Kumar	2000.00	2024-02-01	Paid
Sneha Patel	2800.00	2024-02-10	Paid
Arjun Mehta	4000.00	2024-02-20	Paid
Priya Singh	5500.00	2024-03-01	Paid
Karan Verma	5000.00	2024-03-05	Paid
Ananya Das	7000.00	2024-03-15	Paid
Vikram Rao	9000.00	2024-03-25	Paid
Neha Gupta	9000.00	2024-04-01	Paid
Rahul Nair	12000.00	2024-04-10	Paid
Pooja Iyer	15000.00	2024-04-18	Paid
Aditya Joshi	800.00	2024-04-25	Pending
Meera Kapoor	2200.00	2024-05-01	Paid
Siddharth Roy	7000.00	2024-05-10	Paid
Kavya Nair	2500.00	2024-05-18	Pending
Harsha Reddy	7000.00	2024-05-25	Paid
Tanvi Shah	20000.00	2024-06-01	Paid
Manish Yadav	1200.00	2024-06-10	Pending
Ishita Jain	3000.00	2024-06-20	Paid

20 rows in set (0.0017 sec)

5. Show plan with corresponding members

```
MySQL localhost:3306 ssl gym_membership_management_system SQL -> select p.plan_name, m.member_name from plans p inner join members
on p.plan_id = ms.plan_id inner join members m on ms.member_id = m.member_id;
```

plan_name	member_name
Basic Monthly	Aarav Sharma
Standard Monthly	Diya Reddy
Premium Monthly	Rohan Kumar
Basic Quarterly	Sneha Patel
Standard Quarterly	Arjun Mehta
Premium Quarterly	Priya Singh
Basic Half-Yearly	Karan Verma
Standard Half-Yearly	Ananya Das
Premium Half-Yearly	Vikram Rao
Basic Yearly	Neha Gupta
Standard Yearly	Rahul Nair
Premium Yearly	Pooja Iyer
Student Monthly	Aditya Joshi
Student Quarterly	Meera Kapoor
Student Yearly	Siddharth Roy
Couple Monthly	Kavya Nair
Couple Quarterly	Harsha Reddy
Couple Yearly	Tanvi Shah
Weekend Plan	Manish Yadav
Personal Training Plan	Ishita Jain

20 rows in set (0.0017 sec)

Aggregate

1. Count total members

```
MySQL localhost:3306 ssl gym_membership_management_system SQL -> select count(*) as total_members from members;
```

total_members
20

1 row in set (0.0018 sec)

2. Find average plan price

```
MySQL localhost:3306 ssl gym_membership_management_system SQL> select avg(price) as avg_price from plans;
+-----+
| avg_price |
+-----+
| 5875.000000 |
+-----+
1 row in set (0.0047 sec)
```

3. Calculate total revenue

```
MySQL localhost:3306 ssl gym_membership_management_system SQL> select sum(amount) as total_revenue from payments;
+-----+
| total_revenue |
+-----+
| 117500.00 |
+-----+
1 row in set (0.0016 sec)
```

4. Find highest plan price

```
MySQL localhost:3306 ssl gym_membership_management_system SQL> select max(price) as highest_price from plans;
+-----+
| highest_price |
+-----+
| 20000.00 |
+-----+
1 row in set (0.0042 sec)
```

5. Count members per plan

```
MySQL localhost:3306 ssl gym_membership_management_system SQL> select plan_id, count(member_id) as total_members from memberships
-> Group by plan_id;
+-----+-----+
| plan_id | total_members |
+-----+-----+
| 1 | 1 |
| 2 | 1 |
| 3 | 1 |
| 4 | 1 |
| 5 | 1 |
| 6 | 1 |
| 7 | 1 |
| 8 | 1 |
| 9 | 1 |
| 10 | 1 |
| 11 | 1 |
| 12 | 1 |
| 13 | 1 |
| 14 | 1 |
| 15 | 1 |
| 16 | 1 |
| 17 | 1 |
| 18 | 1 |
| 19 | 1 |
| 20 | 1 |
+-----+-----+
20 rows in set (0.0051 sec)
```

Subqueries

1. Find members without memberships

```
MySQL localhost:3306 ssl gym_membership_management_system SQL> select * from members where member_id not in (
-> select member_id from memberships);
Empty set (0.0044 sec)
```

2. Show plans above average price


```
MySQL localhost:3306 ssl gym_membership_management_system SQL> select * from plans
-> where price > (select avg(price) from plans);
```

plan_id	plan_name	duration	price
8	Standard Half-Yearly	6	7000.00
9	Premium Half-Yearly	6	9000.00
10	Basic Yearly	12	9000.00
11	Standard Yearly	12	12000.00
12	Premium Yearly	12	15000.00
15	Student Yearly	12	7000.00
17	Couple Quarterly	3	7000.00
18	Couple Yearly	12	20000.00

8 rows in set (0.0026 sec)

3. Find members with highest spending

```
MySQL localhost:3306 ssl gym_membership_management_system SQL> SELECT member_id, SUM(amount) AS total_spent
-> FROM Payments
-> GROUP BY member_id
-> ORDER BY total_spent DESC
-> LIMIT 1;
```

member_id	total_spent
18	20000.00

1 row in set (0.0019 sec)

4. Find plans with minimum price

```
MySQL localhost:3306 ssl gym_membership_management_system SQL> select * from plans where price = (select min(price) from plans);
```

plan_id	plan_name	duration	price
13	Student Monthly	1	800.00

1 row in set (0.0017 sec)

5. Show members whose spending is above average

```
MySQL localhost:3306 ssl gym_membership_management_system SQL> select member_id, sum(amount) as total_spent
-> from payments
-> group by member_id
-> having total_spent > (select avg(amount) from payments);
```

member_id	total_spent
8	7000.00
9	9000.00
10	9000.00
11	12000.00
12	15000.00
15	7000.00
17	7000.00
18	20000.00

8 rows in set (0.0020 sec)

Updates & Deletes

1. Update plan price

```
MySQL localhost:3306 ssl gym_membership_management_system SQL > update plans set price = 6000 where plan_id = 1;
Query OK, 1 row affected (0.0089 sec)

Rows matched: 1 Changed: 1 Warnings: 0
MySQL localhost:3306 ssl gym_membership_management_system SQL > select * from plans;
```

plan_id	plan_name	duration	price
1	Basic Monthly	1	6000.00
2	Standard Monthly	1	1500.00
3	Premium Monthly	1	2000.00
4	Basic Quarterly	3	2800.00
5	Standard Quarterly	3	4000.00
6	Premium Quarterly	3	5500.00
7	Basic Half-Yearly	6	5000.00
8	Standard Half-Yearly	6	7000.00
9	Premium Half-Yearly	6	9000.00
10	Basic Yearly	12	9000.00
11	Standard Yearly	12	12000.00
12	Premium Yearly	12	15000.00
13	Student Monthly	1	800.00
14	Student Quarterly	3	2200.00
15	Student Yearly	12	7000.00
16	Couple Monthly	1	2500.00
17	Couple Quarterly	3	7000.00
18	Couple Yearly	12	20000.00
19	Weekend Plan	1	1200.00
20	Personal Training Plan	1	3000.00

```
20 rows in set (0.0015 sec)
```

2. Change member email

```
MySQL localhost:3306 ssl gym_membership_management_system SQL > update members set email = 'newemail@gmail.com'
--> where member_id = 1;
Query OK, 1 row affected (0.0055 sec)

Rows matched: 1 Changed: 1 Warnings: 0
MySQL localhost:3306 ssl gym_membership_management_system SQL > select * from members;
```

member_id	member_name	email	phone	join_date
1	Aarav Sharma	newemail@gmail.com	9876543210	2024-01-10
2	Diya Reddy	diya@gmail.com	9123456780	2024-01-15
3	Rohan Kumar	rohan@gmail.com	9012345678	2024-02-01
4	Sneha Patel	sneha@gmail.com	9988776655	2024-02-10
5	Arjun Mehta	arjun@gmail.com	9871234567	2024-02-20
6	Priya Singh	priya@gmail.com	9765432109	2024-03-01
7	Karan Verma	karan@gmail.com	9654321098	2024-03-05
8	Ananya Das	ananya@gmail.com	9543210987	2024-03-15
9	Vikram Rao	vikram@gmail.com	9432109876	2024-03-25
10	Neha Gupta	neha@gmail.com	9321098765	2024-04-01
11	Rahul Nair	rahul@gmail.com	9210987654	2024-04-10
12	Pooja Iyer	pooja@gmail.com	9109876543	2024-04-18
13	Aditya Joshi	aditya@gmail.com	9098765432	2024-04-25
14	Meera Kapoor	meera@gmail.com	8987654321	2024-05-01
15	Siddharth Roy	siddharth@gmail.com	8876543210	2024-05-10
16	Kavya Nair	kavya@gmail.com	8765432109	2024-05-18
17	Harsha Reddy	harsha@gmail.com	8654321098	2024-05-25
18	Tanvi Shah	tanvi@gmail.com	8543210987	2024-06-01
19	Manish Yadav	manish@gmail.com	8432109876	2024-06-10
20	Ishita Jain	ishita@gmail.com	8321098765	2024-06-20

```
20 rows in set (0.0014 sec)
```

3. Delete inactive members

```

MySQL localhost:3306 ssl gym_membership_management_system SQL > delete from members where member_id not in (
    -> select member_id from memberships );
Query OK, 0 rows affected (0.0055 sec)
MySQL localhost:3306 ssl gym_membership_management_system SQL > select * from members;
+-----+-----+-----+-----+-----+
| member_id | member_name | email | phone | join_date |
+-----+-----+-----+-----+-----+
| 1 | Aarav Sharma | newemail@gmail.com | 9876543210 | 2024-01-10 |
| 2 | Diya Reddy | diya@gmail.com | 9123456780 | 2024-01-15 |
| 3 | Rohan Kumar | rohan@gmail.com | 9012345678 | 2024-02-01 |
| 4 | Sneha Patel | sneha@gmail.com | 9988776655 | 2024-02-10 |
| 5 | Arjun Mehta | arjun@gmail.com | 9871234567 | 2024-02-20 |
| 6 | Priya Singh | priya@gmail.com | 9765432109 | 2024-03-01 |
| 7 | Karan Verma | karan@gmail.com | 9654321098 | 2024-03-05 |
| 8 | Ananya Das | ananya@gmail.com | 9543210987 | 2024-03-15 |
| 9 | Vikram Rao | vikram@gmail.com | 9432109876 | 2024-03-25 |
| 10 | Neha Gupta | neha@gmail.com | 9321098765 | 2024-04-01 |
| 11 | Rahul Nair | rahul@gmail.com | 9210987654 | 2024-04-10 |
| 12 | Pooja Iyer | pooja@gmail.com | 9109876543 | 2024-04-18 |
| 13 | Aditya Joshi | aditya@gmail.com | 9098765432 | 2024-04-25 |
| 14 | Meera Kapoor | meera@gmail.com | 8987654321 | 2024-05-01 |
| 15 | Siddharth Roy | siddharth@gmail.com | 8876543210 | 2024-05-10 |
| 16 | Kavya Nair | kavya@gmail.com | 8765432109 | 2024-05-18 |
| 17 | Harsha Reddy | harsha@gmail.com | 8654321098 | 2024-05-25 |
| 18 | Tanvi Shah | tanvi@gmail.com | 8543210987 | 2024-06-01 |
| 19 | Manish Yadav | manish@gmail.com | 8432109876 | 2024-06-10 |
| 20 | Ishita Jain | ishita@gmail.com | 8321098765 | 2024-06-20 |
+-----+-----+-----+-----+-----+
20 rows in set (0.0017 sec)

```

4. Update trainer details

```

MySQL localhost:3306 ssl gym_membership_management_system SQL > update trainers set phone = '9999999999'
    -> where trainer_id = 1;
Query OK, 1 row affected (0.0065 sec)
Rows matched: 1 Changed: 1 Warnings: 0
MySQL localhost:3306 ssl gym_membership_management_system SQL > select * from trainers;
+-----+-----+-----+-----+
| trainer_id | trainer_name | specialization | phone |
+-----+-----+-----+-----+
| 1 | Rajesh Kumar | Weight Training | 9999999999 |
| 2 | Anita Sharma | Yoga | 9876502345 |
| 3 | Vikram Singh | Cardio Training | 9876503456 |
| 4 | Sneha Reddy | Zumba | 9876504567 |
| 5 | Arjun Patel | Bodybuilding | 9876505678 |
| 6 | Priya Nair | Aerobics | 9876506789 |
| 7 | Karan Mehta | CrossFit | 9876507890 |
| 8 | Neha Gupta | Pilates | 9876508901 |
| 9 | Rohit Verma | Strength Training | 9876509012 |
| 10 | Pooja Das | Yoga Therapy | 9876510123 |
| 11 | Siddharth Roy | HIIT | 9876511234 |
| 12 | Meera Kapoor | Dance Fitness | 9876512345 |
| 13 | Aditya Joshi | Functional Training | 9876513456 |
| 14 | Kavya Iyer | Meditation | 9876514567 |
| 15 | Harsha Rao | Endurance Training | 9876515678 |
| 16 | Tanvi Shah | Kickboxing | 9876516789 |
| 17 | Manish Yadav | Powerlifting | 9876517890 |
| 18 | Ishita Jain | Flexibility Training | 9876518901 |
| 19 | Rahul Khanna | Sports Training | 9876519012 |
| 20 | Divya Agarwal | Personal Training | 9876520123 |
+-----+-----+-----+-----+
20 rows in set (0.0009 sec)

```

5. Delete memberships with expired plans

```
MySQL localhost:3306 ssl gym_membership_management_system SQL > delete from memberships
-> where end_date < '2024-05-20';
Query OK, 4 rows affected (0.0050 sec)
MySQL localhost:3306 ssl gym_membership_management_system SQL > select * from memberships;
```

membership_id	member_id	plan_id	start_date	end_date
5	5	5	2024-02-20	2024-05-20
6	6	6	2024-03-01	2024-06-01
7	7	7	2024-03-05	2024-09-05
8	8	8	2024-03-15	2024-09-15
9	9	9	2024-03-25	2024-09-25
10	10	10	2024-04-01	2025-04-01
11	11	11	2024-04-10	2025-04-10
12	12	12	2024-04-18	2025-04-18
13	13	13	2024-04-25	2024-05-25
14	14	14	2024-05-01	2024-08-01
15	15	15	2024-05-10	2025-05-10
16	16	16	2024-05-18	2024-06-18
17	17	17	2024-05-25	2024-08-25
18	18	18	2024-06-01	2025-06-01
19	19	19	2024-06-10	2024-07-10
20	20	20	2024-06-20	2024-07-20

```
16 rows in set (0.0016 sec)
```

Views

1. Create view for membership details

```
MySQL localhost:3306 ssl gym_membership_management_system SQL > create view memberships_details as
-> select * from memberships;
Query OK, 0 rows affected (0.0090 sec)
MySQL localhost:3306 ssl gym_membership_management_system SQL > select * from memberships_details;
```

membership_id	member_id	plan_id	start_date	end_date
5	5	5	2024-02-20	2024-05-20
6	6	6	2024-03-01	2024-06-01
7	7	7	2024-03-05	2024-09-05
8	8	8	2024-03-15	2024-09-15
9	9	9	2024-03-25	2024-09-25
10	10	10	2024-04-01	2025-04-01
11	11	11	2024-04-10	2025-04-10
12	12	12	2024-04-18	2025-04-18
13	13	13	2024-04-25	2024-05-25
14	14	14	2024-05-01	2024-08-01
15	15	15	2024-05-10	2025-05-10
16	16	16	2024-05-18	2024-06-18
17	17	17	2024-05-25	2024-08-25
18	18	18	2024-06-01	2025-06-01
19	19	19	2024-06-10	2024-07-10
20	20	20	2024-06-20	2024-07-20

```
16 rows in set (0.0047 sec)
```

OR

Best Approach:

Create view for membership details” usually means:

Show complete readable data (with names, not IDs)

```
MySQL localhost:3306 ssl gym_membership_management_system SQL > create view membership_details as
-> select m.member_name, p.plan_name, ms.start_date, ms.end_date
-> from memberships ms
-> inner join members m on ms.member_id = m.member_id
-> inner join plans p on ms.plan_id = p.plan_id;
Query OK, 0 rows affected (0.0162 sec)
MySQL localhost:3306 ssl gym_membership_management_system SQL > select * from membership_details;
```

member_name	plan_name	start_date	end_date
Arjun Mehta	Standard Quarterly	2024-02-20	2024-05-20
Priya Singh	Premium Quarterly	2024-03-01	2024-06-01
Karan Verma	Basic Half-Yearly	2024-03-05	2024-09-05
Ananya Das	Standard Half-Yearly	2024-03-15	2024-09-15
Vikram Rao	Premium Half-Yearly	2024-03-25	2024-09-25
Neha Gupta	Basic Yearly	2024-04-01	2025-04-01
Rahul Nair	Standard Yearly	2024-04-10	2025-04-10
Pooja Iyer	Premium Yearly	2024-04-18	2025-04-18
Aditya Joshi	Student Monthly	2024-04-25	2024-05-25
Meera Kapoor	Student Quarterly	2024-05-01	2024-08-01
Siddharth Roy	Student Yearly	2024-05-10	2025-05-10
Kavya Nair	Couple Monthly	2024-05-18	2024-06-18
Harsha Reddy	Couple Quarterly	2024-05-25	2024-08-25
Tanvi Shah	Couple Yearly	2024-06-01	2025-06-01
Manish Yadav	Weekend Plan	2024-06-10	2024-07-10
Ishita Jain	Personal Training Plan	2024-06-20	2024-07-20

16 rows in set (0.0039 sec)

2. Create view for payment summary

```
MySQL localhost:3306 ssl gym_membership_management_system SQL > create view payment_summary as
-> select member_id, sum(amount) as total_payment
-> from payments group by member_id;
Query OK, 0 rows affected (0.0206 sec)
MySQL localhost:3306 ssl gym_membership_management_system SQL > select * from payment_summary;
```

member_id	total_payment
1	1000.00
2	1500.00
3	2000.00
4	2800.00
5	4000.00
6	5500.00
7	5000.00
8	7000.00
9	9000.00
10	9000.00
11	12000.00
12	15000.00
13	800.00
14	2200.00
15	7000.00
16	2500.00
17	7000.00
18	20000.00
19	1200.00
20	3000.00

20 rows in set (0.0084 sec)

3. Create view for member plans

```
MySQL localhost:3306 ssl gym_membership_management_system SQL> create view member_plans as
-> select m.member_name, p.plan_name
-> from members m
-> inner join memberships ms on m.member_id = ms.member_id
-> inner join plans p on ms.plan_id = p.plan_id;

Query OK, 0 rows affected (0.0118 sec)

MySQL localhost:3306 ssl gym_membership_management_system SQL> select * from member_plans;

+-----+-----+
| member_name | plan_name |
+-----+-----+
| Arjun Mehta | Standard Quarterly |
| Priya Singh | Premium Quarterly |
| Karan Verma | Basic Half-Yearly |
| Ananya Das | Standard Half-Yearly |
| Vikram Rao | Premium Half-Yearly |
| Neha Gupta | Basic Yearly |
| Rahul Nair | Standard Yearly |
| Pooja Iyer | Premium Yearly |
| Aditya Joshi | Student Monthly |
| Meera Kapoor | Student Quarterly |
| Siddharth Roy | Student Yearly |
| Kavya Nair | Couple Monthly |
| Harsha Reddy | Couple Quarterly |
| Tanvi Shah | Couple Yearly |
| Manish Yadav | Weekend Plan |
| Ishita Jain | Personal Training Plan |
+-----+-----+

16 rows in set (0.0065 sec)
```

4. Create view for trainer assignments

```
MySQL localhost:3306 ssl gym_membership_management_system SQL> create view trainer_assignments as
-> select trainer_name, specialization from trainers;

Query OK, 0 rows affected (0.0084 sec)

MySQL localhost:3306 ssl gym_membership_management_system SQL> select * from trainer_assignments;

+-----+-----+
| trainer_name | specialization |
+-----+-----+
| Rajesh Kumar | Weight Training |
| Anita Sharma | Yoga |
| Vikram Singh | Cardio Training |
| Sneha Reddy | Zumba |
| Arjun Patel | Bodybuilding |
| Priya Nair | Aerobics |
| Karan Mehta | CrossFit |
| Neha Gupta | Pilates |
| Rohit Verma | Strength Training |
| Pooja Das | Yoga Therapy |
| Siddharth Roy | HIIT |
| Meera Kapoor | Dance Fitness |
| Aditya Joshi | Functional Training |
| Kavya Iyer | Meditation |
| Harsha Rao | Endurance Training |
| Tanvi Shah | Kickboxing |
| Manish Yadav | Powerlifting |
| Ishita Jain | Flexibility Training |
| Rahul Khanna | Sports Training |
| Divya Agarwal | Personal Training |
+-----+-----+

20 rows in set (0.0085 sec)
```

5. Create view for active memberships

```
MySQL localhost:3306 ssl gym_membership_management_system SQL> create view active_memberships1 as
-> select * from memberships
-> where curdate() between start_date and end_date;

Query OK, 0 rows affected (0.0103 sec)

MySQL localhost:3306 ssl gym_membership_management_system SQL> select * from active_memberships1;

Empty set (0.0029 sec)
```

OR

```
MySQL localhost:3306 ssl gym_membership_management_system SQL > CREATE VIEW active_memberships AS
-> SELECT *
-> FROM Memberships
-> WHERE start_date <= '2024-06-01'
-> AND end_date >= '2024-06-01';

Query OK, 0 rows affected (0.0093 sec)
MySQL localhost:3306 ssl gym_membership_management_system SQL > select * from active_memberships;
```

membership_id	member_id	plan_id	start_date	end_date
6	6	6	2024-03-01	2024-06-01
7	7	7	2024-03-05	2024-09-05
8	8	8	2024-03-15	2024-09-15
9	9	9	2024-03-25	2024-09-25
10	10	10	2024-04-01	2025-04-01
11	11	11	2024-04-10	2025-04-10
12	12	12	2024-04-18	2025-04-18
14	14	14	2024-05-01	2024-08-01
15	15	15	2024-05-10	2025-05-10
16	16	16	2024-05-18	2024-06-18
17	17	17	2024-05-25	2024-08-25
18	18	18	2024-06-01	2025-06-01

12 rows in set (0.0051 sec)

Stored Procedures

1. Insert a New Member

Create a stored procedure to insert a new member.

The procedure should accept:

member_name

email

phone

Join_date

```
delimiter' at line 1
MySQL localhost:3306 ssl gym_membership_management_system SQL > delimiter //
MySQL localhost:3306 ssl gym_membership_management_system SQL > create procedure add_member(in id int,in name varchar(20),
-> in mail varchar(50), in phone_no varchar(15),in j_date date)
-> begin
-> insert into members (member_id,member_name,email,phone,join_date)
-> values(id,name,mail,phone_no,j_date);
-> end //

Query OK, 0 rows affected (0.0088 sec)
MySQL localhost:3306 ssl gym_membership_management_system SQL > delimiter ;
MySQL localhost:3306 ssl gym_membership_management_system SQL > call add_member (21,'Rahul Sharma','rahul123@gmail.com','9876543211','2024-07-01');
Query OK, 1 row affected (0.0057 sec)
MySQL localhost:3306 ssl gym_membership_management_system SQL > select * from members;
```

member_id	member_name	email	phone	join_date
1	Aarav Sharma	newemail@gmail.com	9876543210	2024-01-10
2	Diya Reddy	diya@gmail.com	9123456789	2024-01-15
3	Rohan Kumar	rohan@gmail.com	9012345678	2024-02-01
4	Sneha Patel	sneha@gmail.com	9988776655	2024-02-10
5	Arjun Mehta	arjun@gmail.com	9871234567	2024-02-20
6	Priya Singh	priya@gmail.com	9765432109	2024-03-01
7	Karan Verma	karan@gmail.com	9654321098	2024-03-05
8	Ananya Das	ananya@gmail.com	9543210987	2024-03-15
9	Vikram Rao	vikram@gmail.com	9432109876	2024-03-25
10	Neha Gupta	neha@gmail.com	9321098765	2024-04-01
11	Rahul Nair	rahul@gmail.com	9210987654	2024-04-10
12	Pooja Iyer	pooja@gmail.com	9109876543	2024-04-18
13	Aditya Joshi	aditya@gmail.com	9098765432	2024-04-25
14	Meera Kapoor	meera@gmail.com	8987654321	2024-05-01
15	Siddharth Roy	siddharth@gmail.com	8876543210	2024-05-10
16	Kavya Nair	kavya@gmail.com	8765432109	2024-05-18
17	Harsha Reddy	harsha@gmail.com	8654321098	2024-05-25
18	Tanvi Shah	tanvi@gmail.com	8543210987	2024-06-01
19	Manish Yadav	manish@gmail.com	8432109876	2024-06-10
20	Ishita Jain	ishita@gmail.com	8321098765	2024-06-20
21	Rahul Sharma	rahul123@gmail.com	9876543211	2024-07-01

21 rows in set (0.0018 sec)

2. Update Plan Price

Create a stored procedure to update plan price.

Pass:

plan_id

New_price

By using INOUT :

```
MySQL localhost:3306 ssl gym_membership_management_system SQL > delimiter //
MySQL localhost:3306 ssl gym_membership_management_system SQL > create procedure update_plan_price
-> (inout pid int, inout new_price decimal(10,2))
-> begin
-> update plans set price = new_price where plan_id=pid;
-> end //
Query OK, 0 rows affected (0.0088 sec)
MySQL localhost:3306 ssl gym_membership_management_system SQL > delimiter ;
MySQL localhost:3306 ssl gym_membership_management_system SQL > set @pid=1;
Query OK, 0 rows affected (0.0046 sec)
MySQL localhost:3306 ssl gym_membership_management_system SQL > set @new_price=6500;
Query OK, 0 rows affected (0.0011 sec)
MySQL localhost:3306 ssl gym_membership_management_system SQL > call update_plan_price(@pid,@new_price);
Query OK, 1 row affected (0.0078 sec)
MySQL localhost:3306 ssl gym_membership_management_system SQL > select * from plans;
```

plan_id	plan_name	duration	price
1	Basic Monthly	1	6500.00
2	Standard Monthly	1	1500.00
3	Premium Monthly	1	2000.00
4	Basic Quarterly	3	2800.00
5	Standard Quarterly	3	4000.00
6	Premium Quarterly	3	5500.00
7	Basic Half-Yearly	6	5000.00
8	Standard Half-Yearly	6	7000.00
9	Premium Half-Yearly	6	9000.00
10	Basic Yearly	12	9000.00
11	Standard Yearly	12	12000.00
12	Premium Yearly	12	15000.00
13	Student Monthly	1	800.00
14	Student Quarterly	3	2200.00
15	Student Yearly	12	7000.00
16	Couple Monthly	1	2500.00
17	Couple Quarterly	3	7000.00
18	Couple Yearly	12	20000.00
19	Weekend Plan	1	1200.00
20	Personal Training Plan	1	3000.00

20 rows in set (0.0017 sec)

OR

By using IN:

```
MySQL localhost:3306 ssl gym_membership_management_system SQL > delimiter //
MySQL localhost:3306 ssl gym_membership_management_system SQL > create procedure update_plan_price1 (
    -> in pid int,in new_price decimal(10,2))
    -> begin
    -> update plans set price = new_price where plan_id=pid;
    -> end //
Query OK, 0 rows affected (0.0070 sec)
MySQL localhost:3306 ssl gym_membership_management_system SQL > delimiter ;
MySQL localhost:3306 ssl gym_membership_management_system SQL > call update_plan_price1(1,6500);
Query OK, 0 rows affected (0.0025 sec)
MySQL localhost:3306 ssl gym_membership_management_system SQL > select * from plans;
```

plan_id	plan_name	duration	price
1	Basic Monthly	1	6500.00
2	Standard Monthly	1	1500.00
3	Premium Monthly	1	2000.00
4	Basic Quarterly	3	2800.00
5	Standard Quarterly	3	4000.00
6	Premium Quarterly	3	5500.00
7	Basic Half-Yearly	6	5000.00
8	Standard Half-Yearly	6	7000.00
9	Premium Half-Yearly	6	9000.00
10	Basic Yearly	12	9000.00
11	Standard Yearly	12	12000.00
12	Premium Yearly	12	15000.00
13	Student Monthly	1	800.00
14	Student Quarterly	3	2200.00
15	Student Yearly	12	7000.00
16	Couple Monthly	1	2500.00
17	Couple Quarterly	3	7000.00
18	Couple Yearly	12	20000.00
19	Weekend Plan	1	1200.00
20	Personal Training Plan	1	3000.00

```
20 rows in set (0.0016 sec)
```

3. Display Memberships of a Member

Create a stored procedure to display all memberships of a member.

Pass:

member_id

Return:

All memberships of that member

```
MySQL localhost:3306 ssl gym_membership_management_system SQL > DELIMITER //
MySQL localhost:3306 ssl gym_membership_management_system SQL > CREATE PROCEDURE get_memberships(IN mid INT)
    -> BEGIN
    ->     SELECT * FROM Memberships WHERE member_id = mid;
    -> END //
Query OK, 0 rows affected (0.0087 sec)
MySQL localhost:3306 ssl gym_membership_management_system SQL > DELIMITER ;
MySQL localhost:3306 ssl gym_membership_management_system SQL > call get_memberships(10);
```

membership_id	member_id	plan_id	start_date	end_date
10	10	10	2024-04-01	2025-04-01

```
1 row in set (0.0063 sec)
Query OK, 0 rows affected (0.0063 sec)
```

4. Calculate Total Revenue

Create a stored procedure to calculate total revenue.

The procedure should return:
Total amount from Payments table

Without using any parameter:

```
MySQL localhost:3306 ssl gym_membership_management_system SQL > DELIMITER //
MySQL localhost:3306 ssl gym_membership_management_system SQL > CREATE PROCEDURE total_revenue()
-> BEGIN
->   SELECT SUM(amount) AS revenue FROM Payments;
-> END //
Query OK, 0 rows affected (0.0104 sec)
MySQL localhost:3306 ssl gym_membership_management_system SQL > DELIMITER ;
MySQL localhost:3306 ssl gym_membership_management_system SQL > call total_revenue();
+-----+
| revenue |
+-----+
| 117500.00 |
+-----+
1 row in set (0.0035 sec)
Query OK, 0 rows affected (0.0035 sec)
```

OR

With using OUT parameter:

```
MySQL localhost:3306 ssl gym_membership_management_system SQL > DELIMITER //
MySQL localhost:3306 ssl gym_membership_management_system SQL > CREATE PROCEDURE total_revenue1(OUT total int)
-> BEGIN
->   SELECT SUM(amount) into total FROM Payments;
-> END //
Query OK, 0 rows affected (0.0095 sec)
MySQL localhost:3306 ssl gym_membership_management_system SQL > call total_revenue1(@total)
-> //
Query OK, 1 row affected (0.0030 sec)
MySQL localhost:3306 ssl gym_membership_management_system SQL > select @total //
+-----+
| @total |
+-----+
| 117500 |
+-----+
1 row in set (0.0015 sec)
```

5. List Members of a Plan

Create a stored procedure to list members of a plan.

Pass:

plan_id

Display:

All members enrolled in that plan

```

MySQL localhost:3306 ssl gym_membership_management_system SQL > DELIMITER //
MySQL localhost:3306 ssl gym_membership_management_system SQL > CREATE PROCEDURE members_by_plan(IN pid INT)
-> BEGIN
-> SELECT m.member_name FROM Members m
-> INNER JOIN Memberships ms ON m.member_id = ms.member_id
-> WHERE ms.plan_id = pid;
-> END //
Query OK, 0 rows affected (0.0067 sec)
MySQL localhost:3306 ssl gym_membership_management_system SQL > DELIMITER ;
MySQL localhost:3306 ssl gym_membership_management_system SQL > call members_by_plan(12);
+-----+
| member_name |
+-----+
| Pooja Iyer |
+-----+
1 row in set (0.0030 sec)
Query OK, 0 rows affected (0.0030 sec)

```

Reports / Analysis

Top 5 most popular plans

```

MySQL localhost:3306 ssl gym_membership_management_system SQL > SELECT plan_id, COUNT(*) AS total
-> FROM Memberships
-> GROUP BY plan_id
-> ORDER BY total DESC
-> LIMIT 5;
+-----+-----+
| plan_id | total |
+-----+-----+
| 5 | 1 |
| 6 | 1 |
| 7 | 1 |
| 8 | 1 |
| 9 | 1 |
+-----+-----+
5 rows in set (0.0025 sec)

```

Most active members

```

MySQL localhost:3306 ssl gym_membership_management_system SQL > SELECT member_id, COUNT(*) AS total
-> FROM Memberships
-> GROUP BY member_id
-> ORDER BY total DESC;
+-----+-----+
| member_id | total |
+-----+-----+
| 5 | 1 |
| 6 | 1 |
| 7 | 1 |
| 8 | 1 |
| 9 | 1 |
| 10 | 1 |
| 11 | 1 |
| 12 | 1 |
| 13 | 1 |
| 14 | 1 |
| 15 | 1 |
| 16 | 1 |
| 17 | 1 |
| 18 | 1 |
| 19 | 1 |
| 20 | 1 |
+-----+-----+
16 rows in set (0.0012 sec)

```

Monthly revenue report

```
MySQL localhost:3306 ssl gym_membership_management_system SQL> SELECT MONTH(payment_date) AS month, SUM(amount) AS revenue
-> FROM Payments
-> GROUP BY MONTH(payment_date);
```

month	revenue
1	2500.00
2	8800.00
3	26500.00
4	36800.00
5	18700.00
6	24200.00

6 rows in set (0.0050 sec)

Plan generating highest revenue

```
MySQL localhost:3306 ssl gym_membership_management_system SQL> SELECT ms.plan_id, SUM(pay.amount) AS revenue
-> FROM Payments pay
-> JOIN Memberships ms ON pay.member_id = ms.member_id
-> GROUP BY ms.plan_id
-> ORDER BY revenue DESC
-> LIMIT 1;
```

plan_id	revenue
18	20000.00

1 row in set (0.0027 sec)

Member spending analysis

```
MySQL localhost:3306 ssl gym_membership_management_system SQL> SELECT member_id, SUM(amount) AS total_spent
-> FROM Payments
-> GROUP BY member_id;
```

member_id	total_spent
1	1000.00
2	1500.00
3	2000.00
4	2800.00
5	4000.00
6	5500.00
7	5000.00
8	7000.00
9	9000.00
10	9000.00
11	12000.00
12	15000.00
13	800.00
14	2200.00
15	7000.00
16	2500.00
17	7000.00
18	20000.00
19	1200.00
20	3000.00

20 rows in set (0.0017 sec)

Trainer performance report

```
MySQL localhost:3306 ssl gym_membership_management_system SQL> SELECT trainer_name, specialization FROM Trainers;
```

trainer_name	specialization
Rajesh Kumar	Weight Training
Anita Sharma	Yoga
Vikram Singh	Cardio Training
Sneha Reddy	Zumba
Arjun Patel	Bodybuilding
Priya Nair	Aerobics
Karan Mehta	CrossFit
Neha Gupta	Pilates
Rohit Verma	Strength Training
Pooja Das	Yoga Therapy
Siddharth Roy	HIIT
Meera Kapoor	Dance Fitness
Aditya Joshi	Functional Training
Kavya Iyer	Meditation
Harsha Rao	Endurance Training
Tanvi Shah	Kickboxing
Manish Yadav	Powerlifting
Ishita Jain	Flexibility Training
Rahul Khanna	Sports Training
Divya Agarwal	Personal Training

```
20 rows in set (0.0028 sec)
```